claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-
a027b2ec6c277c691.jsonl`
- SHA-256: `f676d8ea67c08e403daa382959fddd06923ae79f4b4797ef0562f08a3f86c41a`
- Source modified: `2026-07-06T06:14:16+00:00`
- Imported at: `2026-07-08T16:00:12+00:00`
- project: `wf_eb99bd9f-440`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T06:09:54.705Z)

```
You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch
feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral
domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py,
src/tg_video_filter/models.py,
tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:
- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite;
  use db._apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and
  db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram
show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest
  434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which
is
  NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No
praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe
first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow
trigger),
  medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact
code at the
cited lines, trace the real control flow, and if useful run a repro with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if
the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return
REFUTED if
the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff).
Return
PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension adr_conformance):
title: Option 1A drops Tier-2 re-upload dedup whenever document_id is present ? the ADR/README
'no dedup-quality loss' claim is false as implemented
severity(claimed): high
file:line: src/tg_video_filter/db.py:285
summary: compute_dedup_key returns tg:doc:{document_id} and never consults the fingerprint when
```

document_id is present, so a re-upload that gets a NEW document.id but identical bytes is no longer deduped ? the exact case ADR 0011 rejects Option 1C for, silently inherited by 1A.
failure_scenario: Account 111 has video A (document_id=1000, size=5MB,dur=60,h=1080,w=1920) already stored. The same video is re-encoded/re-uploaded to a different channel; Telegram assigns document_id=2000 but the four fingerprint fields are identical. Ingest: Tier-0 (msg) misses, Tier-1 (get_video_by_document_id 2000) misses because it's a new id, Tier-2 (get_video_by_content_fingerprint 5MB/60/1080/1920) now ALSO misses because the stored row's dedup_key is 'tg:doc:1000' (the fp branch was never taken at insert), so a tg:fp lookup finds nothing. insert_video(v2) then succeeds ? the row's key is 'tg:doc:2000', which does not collide with 'tg:doc:1000'. Result: the same video is delivered twice, contradicting README's 'Never forwards the same video twice' and ADR 0011 line 366-367 'generalizes ADR 0006 without losing any dedup quality'.
evidence: Repro (venv) ? v1 on main catches it, PR does not:
  v1 (main standalone): 'v1 Tier-2 fingerprint hit? True -> (1,)'
  PR: 'Tier-1 (doc_id=2000) hit? False' / 'Tier-2 (fingerprint) hit? False' / 'insert_video(v2 re-upload) SUCCEEDED -> DUPLICATE ROW admitted' with stored keys 'tg:doc:1000' and 'tg:doc:2000'.
Code: db.py:286-287 returns tg:doc: before the fingerprint branch at db.py:290; get_content_item_by_dedup_key (db.py:1008) only matches an exact key, and the fp-shaped key was never stored. v1 ingest ran Tier-1 and Tier-2 as INDEPENDENT sequential lookups (telethon_client.py:191-236; get_video_by_content_fingerprint on main is a standalone size/dur/h/w query). ADR 0011:113-119 rejects 1C precisely for 're-encodes and re-uploads the same video to a different channel'; 1A shares that loss because Telegram videos almost always carry a document.id, so branch 1 always wins.
recommendation: Either (a) store BOTH a doc key and a fingerprint key per item (e.g. a second unique dedup column / a dedup_keys side table) and check both at ingest, or (b) have the ingest path additionally probe the fingerprint key even when document_id is set. Correct the ADR 0011 'without losing any dedup quality' claim and the README 'three-tier'/'never twice' wording to reflect the actual single-key behaviour, since as shipped the fingerprint tier is dead whenever document_id exists.

2. assistant (2026-07-06T06:09:58.668Z)

I'll verify this finding by reading the actual code at the cited lines and tracing the control flow.

3. user (2026-07-06T06:09:59.651Z)

```
270           1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
271           2. Else if **all four** fingerprint fields (size, duration, height, width)
272              are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273           3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274              ``source_id`` is required because Telegram ``message_id`` is only
275              unique within a channel).
276         - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
277           ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278           providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279
280         Raises ``ValueError`` if the fallback branch is reached without the fields
281         it requires (``source_id`` + ``provider_message_id``) ? a degenerate
282         ``dedup_key`` is worse than a loud failure at ingestion time.
283         """
284         p = provider.lower()
285         if p == "telegram":
286             if document_id is not None and document_id != "":
287                 return f"tg:doc:{document_id}"
288             # All four fingerprint fields are required to avoid a degenerate
289             # ``tg:fp:0:0:0:0`` key when some metadata is missing.
290             if None not in (size_bytes, duration_s, height, width):
291                 return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292             # Tier-0 fallback.
293             if source_id is None or provider_message_id is None:
294                 raise ValueError(
295                     "Telegram dedup_key fallback requires source_id and provider_message_id
"
```

```
                    "when document_id and the full fingerprint are unavailable"
                )
            return f"tg:msg:{source_id}:{provider_message_id}"
        # Future-proof default for providers without a dedicated branch.
        if source_id is None or provider_message_id is None:
            raise ValueError(
                f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
            )
        return f"{p}:msg:{source_id}:{provider_message_id}"


    async def _apply_migrations(conn: aiosqlite.Connection) -> None:
        """Run every registered migration whose version > current ``user_version``."""
        async with conn.execute("PRAGMA user_version") as cur:
            row = await cur.fetchone()
        current = int(row[0]) if row else 0

        for version, fn in _MIGRATIONS:
            if version <= current:
                continue
            await fn(conn)
            await conn.execute(f"PRAGMA user_version = {version}")


    async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
        """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
        async with conn.execute(f"PRAGMA table_info({table})") as cur:
            rows = await cur.fetchall()
        return any(row[1] == column for row in rows)


    @_migration(1)
    async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
        """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
```

4. user (2026-07-06T06:10:00.637Z)

```
      derivation rules need their own unit tests.

    #### Option 1B: content hash (SHA-256 of downloaded bytes)

    `dedup_key = sha256(downloaded_content)`. Cryptographically strong,
    provider-agnostic, catches re-encodes that 1A misses.

    **Rejected** ? violates ADR 0003 (metadata-first, no download in the hot
    path). Downloading every incoming video to hash it would multiply bandwidth
    and latency by orders of magnitude. Could be added later as an optional
    deep-inspect stage (deferred to Track F per #31).

    #### Option 1C: provider-assigned only (drop Tier 2)

    `dedup_key = "tg:doc:{document_id}"` always; no metadata fingerprint
    fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
    `document.id` ? a real and common case (someone re-encodes and re-uploads
    the same video to a different channel).

    **Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly,
and honors the no-download constraint.

    ---

    ### Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?

    The codebase already has `filter_config.config_json` (one row per account,
```

```
126     holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The
127     umbrella introduces a `Policy` concept. They overlap. Two options:
128
129     #### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
130
131     `policies` is a new table; `filter_config` is migrated into it. One account
132     can own multiple policies (enables per-route filter overrides from ADR
133     0009), with a `DEFAULT policy` per account for the no-rules-configured case.
134
135     ```
136     policies (
137         id           INTEGER PRIMARY KEY AUTOINCREMENT,
138         account_id   INTEGER NOT NULL REFERENCES accounts(id),
139         name         TEXT NOT NULL,                     -- e.g. "default", "reggae-route"
```

5. assistant (2026-07-06T06:10:03.580Z)

The core claim hinges on whether the ingest path stores a fingerprint key when document_id is present, and whether the fingerprint tier is a separate stored lookup. Let me examine the insert path and the lookup functions.

6. user (2026-07-06T06:10:05.756Z)

```
254:def compute_dedup_key(
953:async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) -> ContentItem:
1008:async def get_content_item_by_dedup_key(
1137:async def get_video_by_document_id(
1151:async def get_video_by_content_fingerprint(
1184:async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
```

7. user (2026-07-06T06:10:06.837Z)

```
Found 4 files
src\tg_video_filter\db.py
src\tg_video_filter\models.py
src\tg_video_filter\telethon_client.py
src\tg_video_filter\backfill.py
```

8. assistant (2026-07-06T06:10:12.267Z)

The key question: is the ingest path in this PR still using the v1 three-tier sequential lookup (get_video_by_document_id + get_video_by_content_fingerprint), or has it switched to the single compute_dedup_key path? The finding claims the fingerprint tier is "dead". Let me check what telethon_client.py actually calls, and whether v1's get_video_by_content_fingerprint still exists and works.

9. user (2026-07-06T06:10:13.387Z)

```
1137    async def get_video_by_document_id(
1138        conn: aiosqlite.Connection, user_id: int, document_id: int
1139    ) -> VideoRecord | None:
1140        """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141        dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142        item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143        if item is None:
1144            return None
1145        src = await get_source_by_id(conn, item.source_id)
1146        if src is None:
1147            return None
1148        return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151    async def get_video_by_content_fingerprint(
1152        conn: aiosqlite.Connection,
```

```
        user_id: int,
        size_bytes: int,
        duration_s: int,
        height: int,
        width: int,
    ) -> VideoRecord | None:
        """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
        dedup_key = compute_dedup_key(
            provider="telegram",
            size_bytes=size_bytes,
            duration_s=duration_s,
            height=height,
            width=width,
        )
        item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
        if item is None:
            return None
        src = await get_source_by_id(conn, item.source_id)
        if src is None:
            return None
        return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


    async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
        """Resolve a source by its surrogate PK (used by the legacy video shims)."""
        conn.row_factory = aiosqlite.Row
        async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
            row = await cur.fetchone()
        return _row_to_source(row) if row else None


    async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
        """Deprecated shim: insert into ``content_items`` (ADR 0011).

        Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
        if absent), computes the provider-neutral ``dedup_key`` via
        :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
        ``VideoRecord`` shape is reconstructed from the inserted row.
        """
        src = await upsert_source(
            conn,
            Source(
                account_id=video.user_id,
                provider="telegram",
                provider_source_id=str(video.channel_id),
                kind="channel",
                watched=True,
            ),
        )
        assert src.id is not None  # upsert_source populates the surrogate PK
        item = ContentItem(
            account_id=video.user_id,
            source_id=src.id,
            provider="telegram",
            provider_message_id=str(video.message_id),
            content_type="video",
            document_id=str(video.document_id) if video.document_id is not None else None,
            duration_s=video.duration_s,
            width=video.width,
            height=video.height,
            size_bytes=video.size_bytes,
            mime_type=video.mime_type,
            dedup_key=compute_dedup_key(
                provider="telegram",
                document_id=(str(video.document_id) if video.document_id is not None else
```

```
None),
                source_id=src.id,
                provider_message_id=str(video.message_id),
                size_bytes=video.size_bytes,
                duration_s=video.duration_s,
                height=video.height,
                width=video.width,
            ),
            matched=video.matched,
            seen_at=video.seen_at,
        )
        item = await insert_content_item(conn, item)
        return _content_item_to_video(item, src, video.user_id, video.channel_id)


    async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
        """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).

        Per-destination delivery state moves to ``delivery_attempts`` (Track C,
        #28). For Phase 1 (before Track C lands) there is no equivalent column on
        ``content_items`` ? the ``matched`` flag alone is sufficient because v1
        delivery still targets the single ``delivery_target`` and retried delivery
        is harmless (forwards are idempotent). This shim is retained so existing
        call sites in ``delivery.py`` keep working without a flag day.
        """
        # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
        # The row's ``matched`` flag (set at insert time) is the surviving signal.
        return None


    def _content_item_to_video(
        item: ContentItem, src: Source, user_id: int, channel_id: int
    ) -> VideoRecord:
        """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.

        ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
        per-destination delivery state moves to Track C). Existing callers read it
        only to decide whether to skip re-delivery; since forwards are idempotent
        the ``False`` default is safe for Phase 1.
        """
```

10. user (2026-07-06T06:10:14.680Z)

```
9:      -> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
193:            existing = await db.get_video_by_document_id(conn, user.id, document_id)
214:            existing = await db.get_video_by_content_fingerprint(
263:            inserted = await db.insert_video(conn, record)
```

11. assistant (2026-07-06T06:10:19.035Z)

Now I have the full picture. The finding's core claim is architecturally correct: `get_video_by_content_fingerprint` (the Tier-2 shim) now internally calls `compute_dedup_key` with only the fingerprint fields and does an exact `get_content_item_by_dedup_key` lookup. Since the stored row (with document_id present) got dedup_key `tg:doc:1000`, a `tg:fp:...` lookup will never match it. Let me confirm the ingest flow and verify this isn't intercepted elsewhere, and check what telethon_client does between Tier-1 and Tier-2.

12. user (2026-07-06T06:10:20.539Z)

```
185
186             # Tier 0: same (user, channel, message) instance ? already processed.
187             existing = await db.get_video(conn, user.id, channel_id, message_id)
188             if existing is not None:
189                 return
```

```

            # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
            if document_id is not None:
                existing = await db.get_video_by_document_id(conn, user.id, document_id)
                if existing is not None:
                    log.info(
                        "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                        "original_channel=%s original_msg=%s",
                        user.id,
                        document_id,
                        channel_id,
                        message_id,
                        existing.channel_id,
                        existing.message_id,
                    )
                    return

            # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
            if (
                meta.size_bytes is not None
                and meta.duration_s is not None
                and meta.height is not None
                and meta.width is not None
            ):
                existing = await db.get_video_by_content_fingerprint(
                    conn,
                    user.id,
                    meta.size_bytes,
                    meta.duration_s,
                    meta.height,
                    meta.width,
                )
                if existing is not None:
                    log.info(
                        "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
                        "channel=%s msg=%s original_channel=%s original_msg=%s",
                        user.id,
                        meta.size_bytes,
                        meta.duration_s,
                        meta.height,
                        meta.width,
                        channel_id,
                        message_id,
                        existing.channel_id,
                        existing.message_id,
                    )
                    return

            # Lazily record the channel (best-effort; failures are non-fatal).
            try:
                await db.upsert_channel(
                    conn,
                    Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
                )
            except Exception:
                log.warning(
                    "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
                )

            result = engine_holder[ENGINE_KEY].evaluate(meta)
            record = VideoRecord(
                user_id=user.id,
                channel_id=channel_id,
```

```
253                 message_id=message_id,
254                 document_id=document_id,
255                 duration_s=meta.duration_s,
256                 width=meta.width,
257                 height=meta.height,
258                 size_bytes=meta.size_bytes,
259                 mime_type=meta.mime_type,
260                 matched=result.matched,
261             )
262             try:
263                 inserted = await db.insert_video(conn, record)
264             except sqlite3.IntegrityError:
```

13. user (2026-07-06T06:10:21.372Z)

```
1008    async def get_content_item_by_dedup_key(
1009        conn: aiosqlite.Connection, account_id: int, dedup_key: str
1010    ) -> ContentItem | None:
1011        """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1012
1013        Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014        a single query: the ``dedup_key`` already encodes whichever tier was
1015        applicable at ingestion time.
1016        """
1017        conn.row_factory = aiosqlite.Row
1018        async with conn.execute(
1019            "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1020            (account_id, dedup_key),
1021        ) as cur:
1022            row = await cur.fetchone()
1023        return _row_to_content_item(row) if row else None
1024
1025
1026    # --- Policy repos (ADR 0011, Option 2A) -----------------------------------
1027
1028
1029    async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1030        """Insert or update a policy row by ``(account_id, name)``.
1031
1032        Exactly one policy per account should carry ``is_default=True``; callers
1033        are responsible for flipping the previous default before flipping a new
1034        one (enforced by a partial unique index landed by future work, not in the
1035        schema ? SQLite ``CHECK`` can't express "at most one per group").
1036        """
1037        await conn.execute(
1038            """INSERT INTO policies (account_id, name, config_json, is_default,
1039                                     created_at, updated_at)
1040               VALUES (?, ?, ?, ?, ?, ?)
1041               ON CONFLICT(account_id, name) DO UPDATE SET
1042                 config_json=excluded.config_json,
1043                 is_default=excluded.is_default,
1044                 updated_at=excluded.updated_at""",
1045            (
1046                policy.account_id,
1047                policy.name,
```

14. assistant (2026-07-06T06:10:27.903Z)

The finding is architecturally confirmed by reading. But `telethon_client.py` is NOT in this PR's diff (only db.py, models.py, tests). The behavioral change ships through the deprecated shims in db.py which are in the diff. The question is whether the shims changed the semantics. Let me verify v1's `get_video_by_content_fingerprint` on main did an independent SQL query, and run the actual repro to confirm the real behavioral regression ships via the db.py shims.

15. user (2026-07-06T06:10:32.932Z)

```
640:async def get_video_by_content_fingerprint(
641-    conn: aiosqlite.Connection,
642-    user_id: int,
643-    size_bytes: int,
644-    duration_s: int,
645-    height: int,
646-    width: int,
647-) -> VideoRecord | None:
648-    """Tier-2 content dedup: metadata fingerprint match ? probably same video.
649-
650-    Catches re-uploads where Telegram assigns a new ``document.id`` but the
651-    file bytes are identical. Uses (size_bytes, duration_s, height, width)
652-    as a composite key ? the odds of two different videos sharing all four
653-    values are negligible for this use case.
654-    """
655-    conn.row_factory = aiosqlite.Row
656-    async with conn.execute(
657-        """SELECT * FROM videos
658-           WHERE user_id = ?
659-             AND size_bytes = ?
660-             AND duration_s = ?
661-             AND height = ?
662-             AND width = ?
663-           LIMIT 1""",
664-        (user_id, size_bytes, duration_s, height, width),
665-    ) as cur:
--
766:    "get_video_by_content_fingerprint",
767-    "insert_video",
768-    "mark_forwarded",
769-    "load_filter_config",
770-    "save_filter_config",
771-    "list_videos_for_user",
772-]
773-
774-
775-def __dir__() -> Iterable[str]:
776-    return __all__
```

16. assistant (2026-07-06T06:10:36.372Z)

Confirmed: v1's `get_video_by_content_fingerprint` on main did a direct SQL match on the four fingerprint columns regardless of whether the stored row had a document_id. So v1 would catch the re-upload; the PR's shim will not. This is a genuine behavioral regression that ships through the db.py shim (which IS in the diff). Let me run the repro to be certain.

17. assistant (2026-07-06T06:10:45.457Z)

```
import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import VideoRecord

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, __import__("tg_video_filter.models",
fromlist=["Account"]).Account(user_id=111, phone="x")) if hasattr(db, "upsert_account") else
None
    # seed original video with document_id=1000
    v1 = VideoRecord(user_id=111, channel_id=555, message_id=1, document_id=1000,
                     duration_s=60, width=1920, height=1080, size_bytes=5_000_000, matched=True)
    await db.insert_video(conn, v1)
    await conn.commit()
```

```
    # Re-upload: new document_id=2000, identical fingerprint, different channel
    t1 = await db.get_video_by_document_id(conn, 111, 2000)
    print("Tier-1 (doc_id=2000) hit?", t1 is not None)
    t2 = await db.get_video_by_content_fingerprint(conn, 111, 5_000_000, 60, 1080, 1920)
    print("Tier-2 (fingerprint) hit?", t2 is not None)

    # stored dedup keys
    async with conn.execute("SELECT dedup_key FROM content_items") as cur:
        rows = await cur.fetchall()
    print("stored keys:", [r[0] for r in rows])

    v2 = VideoRecord(user_id=111, channel_id=777, message_id=2, document_id=2000,
                     duration_s=60, width=1920, height=1080, size_bytes=5_000_000, matched=True)
    try:
        await db.insert_video(conn, v2)
        await conn.commit()
        print("insert_video(v2 re-upload) SUCCEEDED -> DUPLICATE ROW admitted")
    except Exception as e:
        print("insert blocked:", type(e).__name__, e)
    async with conn.execute("SELECT dedup_key FROM content_items") as cur:
        rows = await cur.fetchall()
    print("final keys:", [r[0] for r in rows])

asyncio.run(main())
```

18. user (2026-07-06T06:10:45.457Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

19. user (2026-07-06T06:12:57.227Z)

```
Command running in background with ID: byf88ybok. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\byf88ybok.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

20. user (2026-07-06T06:13:05.672Z)

```
Legacy videos (videos_deprecated): 2
sources rows: 1
content_items rows (migrated): 1
*** DATA LOSS: 1 video(s) silently dropped by the JOIN ***
content_item for document_id 222000 (the orphan-channel video) -> None
```

21. assistant (2026-07-06T06:13:08.009Z)

```
That output is from a different cached script/module. Let me check the background task output
which ran my actual repro.
```

22. user (2026-07-06T06:13:08.761Z)

```
1       Traceback (most recent call last):
2         File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 41, in
<module>
3           asyncio.run(main())
4           ~~~~~~~~~~~^^^^^^^^
5         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 195, in run
6           return runner.run(main)
7                  ~~~~~~~~~~^^^^^^
8         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 118, in run
9           return self._loop.run_until_complete(task)
```

```
10                ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
11        File
"C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725,
in run_until_complete
12          return future.result()
13                 ~~~~~~~~~~~~~^^
14        File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 17, in main
15          await db.insert_video(conn, v1)
16        File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 1192, in
insert_video
17          src = await upsert_source(
18                ^^^^^^^^^^^^^^^^^^^^
19          ...<8 lines>...
20          )
21          ^
22        File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 904, in
upsert_source
23          await conn.execute(
24          ...<18 lines>...
25          )
26        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 223, in execute
27          cursor = await self._execute(self._conn.execute, sql, parameters)
28                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
29        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 160, in _execute
30          return await future
31                 ^^^^^^^^^^^^
32        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 63, in _connection_worker_thread
33          result = function()
34      sqlite3.IntegrityError: FOREIGN KEY constraint failed
35
```

23. assistant (2026-07-06T06:13:11.849Z)

Need an account row first (FK constraint). Let me check the account seeding helper signature and fix.

24. user (2026-07-06T06:13:12.932Z)

```
src\tg_video_filter\db.py:730:async def upsert_account(conn: aiosqlite.Connection, account:
Account) -> Account:
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-731-    """Insert or update an
account row; returns the persisted Account."""
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-732-    await conn.execute(
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-733-        """INSERT INTO accounts
(id, telegram_id, provider, display_name, created_at)
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-734-           VALUES (?, ?, ?, ?, ?)
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-735-           ON CONFLICT(id) DO
UPDATE SET
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py-736-
telegram_id=excluded.telegram_id,
--
src\tg_video_filter\models.py:35:class Account(BaseModel):
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-36-    """A product identity ?
the owner of config, channels, routes, and content.
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-37-
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-38-    Decoupled from any
specific provider's session (ADR 0010). ``telegram_id``
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-39-    is the bridge used by the
Bot API front-door (``from.id``) and by the
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-40-    legacy userbot path; it is
nullable so future providers that don't have a
```

```
C:\Users\avidu\Projects\Telegram\src\tg_video_filter\models.py-41-    Telegram id (e.g.
WhatsApp) can still own an ``Account``.
```

25. user (2026-07-06T06:13:16.696Z)

```
35      class Account(BaseModel):
36          """A product identity ? the owner of config, channels, routes, and content.
37
38          Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39          is the bridge used by the Bot API front-door (``from.id``) and by the
40          legacy userbot path; it is nullable so future providers that don't have a
41          Telegram id (e.g. WhatsApp) can still own an ``Account``.
42          """
43
44          model_config = ConfigDict(frozen=True)
45
46          id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47          telegram_id: int | None = Field(
48              default=None,
49              description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
50          )
51          provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52          display_name: str | None = None
53          created_at: datetime = Field(default_factory=_utcnow)
54
```

26. user (2026-07-06T06:13:28.078Z)

```
Legacy videos (videos_deprecated): 2
sources rows: 1
content_items rows (migrated): 1
*** DATA LOSS: 1 video(s) silently dropped by the JOIN ***
content_item for document_id 222000 (the orphan-channel video) -> None
```

27. assistant (2026-07-06T06:13:31.505Z)

That output is not from my script ? it's stale/cached output printing something about videos_deprecated. My script never prints "Legacy videos". There must be a conftest or module-level import doing this, or the output file is stale. Let me run with a unique marker and verify.

28. user (2026-07-06T06:13:36.788Z)

```
=== RUNNING FRESH ===
MARKER-XYZ
import asyncio, aiosqlite
import sys; sys.path.insert(0, "src"); from tg_video_filter import db

async def main():
    conn = await aiosqlite.connect(":memory:")
```

29. assistant (2026-07-06T06:13:40.890Z)

The script file was overwritten by a different process/session (concurrent session hazard flagged in my instructions). The current repro.py head differs from what I wrote. Let me write a fresh script with a unique name.

30. user (2026-07-06T06:13:53.387Z)

```
REUP Tier-1(doc=2000) hit? False
REUP Tier-2(fp) hit? False
```

```
REUP keys after v1: ['tg:doc:1000']
REUP insert v2 SUCCEEDED -> duplicate admitted
REUP final keys: ['tg:doc:1000', 'tg:doc:2000']
Exception in thread Thread-1 (_connection_worker_thread):
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 66, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_result, future, result)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in
_bootstrap_inner
    self.run()
    ~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in
run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 75, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_exception, future, e)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

31. assistant (2026-07-06T06:14:05.295Z)

The repro confirms the finding exactly (the trailing event-loop-closed noise is just cleanup, not a functional error). The re-upload with a new document_id and identical fingerprint:
- Tier-1 misses (new doc id)
- Tier-2 fingerprint misses (stored row's key is `tg:doc:1000`, fp lookup builds `tg:fp:...` which doesn't match)
- insert_video succeeds -> duplicate admitted, final keys `['tg:doc:1000', 'tg:doc:2000']`

v1 on main would have caught this via the direct SQL fingerprint match. The regression ships through the db.py shims which ARE in this PR's diff.

Now assessing severity. The finding claims "high". This is a real regression, but let me weigh it honestly against the finding's own framing. The claimed defect is genuine but it's fundamentally the deliberate ADR design choice (Option 1A), not an accidental bug. The finding correctly notes the ADR/README claim of "no dedup-quality loss" is the false part. The trigger (re-encode/re-upload getting a new document_id with byte-identical fingerprint fields) is real but relatively narrow ? Telegram forwards/cross-posts keep the same document_id (caught by Tier-1); only genuine re-encodes/re-uploads produce a new one. It's a real dedup-quality regression versus v1 with a narrow but realistic trigger, and no data loss/corruption. That matches "high" (real bug, narrow trigger) ? arguably medium since it's a documented-design tradeoff, but the behavior genuinely regresses v1's guarantee and the ADR explicitly promised it wouldn't.

32. user (2026-07-06T06:14:16.333Z)

```
Structured output provided successfully
```